

Redux Toolkit – Counter Example (From Scratch)

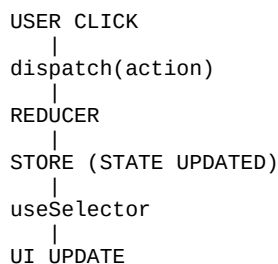
1. What is State?

State means data that can change over time. In a counter application, the state is the number value that increases or decreases.

2. Why Do We Need Redux Toolkit?

When multiple components need the same data, managing state becomes difficult. Redux Toolkit provides a single central place (store) where state lives, so all components can access it easily.

3. Redux Toolkit Big Picture



4. Counter App Requirements

- 1 Display counter value
- 2 Increment counter
- 3 Decrement counter
- 4 Reset counter

5. Counter Slice

A slice is a combination of state and reducers for a specific feature. Here, the counter slice manages counter-related logic.

```
const counterSlice = createSlice({
  name: "counter",
  initialState: { value: 0 },
  reducers: {
    increment(state) {
      state.value += 1;
    },
    decrement(state) {
      state.value -= 1;
    },
    reset(state) {
      state.value = 0;
    }
  }
})
```

```
});
```

6. Store

The store holds the entire application state. Redux Toolkit uses `configureStore` to simplify setup.

```
const store = configureStore({
  reducer: {
    counter: counterReducer
  }
});
```

7. Provider

Provider connects Redux with React. It makes the store available to all components.

8. Dispatching Actions

Dispatch sends actions to the store to request a state change.

```
dispatch(increment());
dispatch(decrement());
dispatch(reset());
```

9. Selecting State

`useSelector` is used to read data from the store.

```
const count = useSelector(state => state.counter.value);
```

10. Mental Model for Students

STORE	-> Brain
SLICE	-> Feature
REDUCER	-> Rules
ACTION	-> Event
DISPATCH	-> Trigger
SELECTOR	-> Reader

11. Final Summary

Redux Toolkit makes state management simple, predictable, and scalable. In the counter example, all state logic is handled in one place, making the application easy to understand and maintain.